



Universidade do Minho

Mestrado em Cibersegurança

Laboratório de Cibersegurança 1

Carlos Daniel Silva Fernandes
(PG59783)

Pedro Augusto Ennes de Martino Camargo
(PG59791)

Luís Filipe Pinheiro Silva
(PG59790)

6 de maio de 2026

Conteúdo

1	Executive Summary	3
2	Scope and Methodology	4
2.1	Pre engagement	4
2.2	Scope	4
2.3	Rules of Engagement	4
2.4	Testing Methodology	5
2.5	Limitations	5
3	Phase A Findings	6
3.1	SQL Injection in API Endpoint	6
3.1.1	Root Cause	6
3.1.2	Impact Analysis	6
3.1.3	Evidence	6
3.1.4	Remediation Recommendations	7
3.2	Broken Access Control	8
3.2.1	Root Cause	8
3.2.2	Impact Analysis	8
3.2.3	Evidence	8
3.2.4	Remediation Recommendations	9
3.3	2FA Bypass via TOTP Secret Exposure	10
3.3.1	Root Cause	10
3.3.2	Impact Analysis	10
3.3.3	Evidence	10
3.3.4	Remediation Recommendations	11
3.4	JWT Token Forgery	11
3.4.1	Root Cause	12
3.4.2	Impact Analysis	12
3.4.3	Evidence	12
3.4.4	Remediation Recommendations	13
4	Phase B Findings	14
4.1	Attack Plan	14
4.1.1	Brute Force Testing Plan	14
4.1.2	Cross-Site Request Forgery Testing Plan	14
4.1.3	SQL Injection Testing Plan	15
4.1.4	Weak Session ID Testing Plan	15
4.2	Summary of Findings	15
4.3	Finding B-01: Vulnerable Login	15
4.4	Finding B-02: Cross-Site Request Forgery	17
4.5	Finding B-03: SQL Injection	20
4.6	Finding B-04: Weak Session IDs	23
5	Cross-Phase Analysis	26
6	Conclusion	28

Capítulo 1

Executive Summary

This report presents the results of a controlled penetration testing exercise performed against two intentionally vulnerable web applications: **OWASP Juice Shop** and the **Damn Vulnerable Web Application (DVWA)**. The objective was to identify, validate, and document common web application security weaknesses using structured testing approaches based on PTES and OWASP WSTG. All activities were conducted in a safe laboratory environment and focused on understanding risk, evidence collection, and practical remediation.

The assessment identified several high-impact vulnerabilities that could allow an attacker to access sensitive data, impersonate users, modify application state, or bypass intended security controls. In Phase A, the most significant issues included SQL injection, broken access control, exposure of two-factor authentication secrets, and JWT token forgery. These findings show that the application trusted user-controlled input, identifiers, secrets, and authentication claims without sufficient server-side validation.

In Phase B, similar techniques were adapted to DVWA's medium security level. Although additional controls were present, they mostly increased the difficulty of exploitation rather than removing the underlying weaknesses. The confirmed findings included vulnerable login behaviour, cross-site request forgery, SQL injection, and weak session identifiers. This demonstrated that client-side restrictions, simple referrer checks, login delays, and predictable session values are not enough to protect an application when the server does not enforce strong security decisions.

Overall, the engagement shows that effective security depends on addressing root causes rather than relying on superficial barriers. The highest-priority recommendations are to use parameterised queries, enforce strict server-side authorisation, protect authentication and 2FA secrets, validate JWT signatures securely, generate strong unpredictable session identifiers, and implement action-specific CSRF protections. Applying these measures would significantly reduce the likelihood of account compromise, data exposure, and unauthorised actions.

Capítulo 2

Scope and Methodology

This engagement was performed as a controlled penetration testing exercise in a local laboratory environment. The objective was to identify, validate, document, and explain realistic web application weaknesses without damaging the target systems.

2.1 Pre engagement

The applications targeted in this engagement were the **OWASP Juice Shop** and the **Damn Vulnerable Web Application (DVWA)**. Both applications are intentionally vulnerable web applications designed for security training and testing purposes.

In this report, we document the findings from both applications, following the PTES and WSTG frameworks. The report includes the activities performed, confirmed findings, evidence collected, and recommended remediation actions.

We used **Burp Suite**, **wordlists**, **Postman**, Docker, browser developer tools, and manual testing techniques to inspect requests, test payloads, and confirm impact.

Furthermore, it is important to note that all testing was conducted within the authorised local scope and according to ethical and legal requirements.

2.2 Scope

The scope was divided into two phases. **Phase A** focused on OWASP Juice Shop, accessed through a local Docker deployment at `http://localhost:3000`. This phase was used to practise structured web exploitation and evidence collection in a guided environment. **Phase B** focused on DVWA running locally at `http://localhost:80`, with the DVWA security level set to **Medium**.

Testing was limited to these two local applications and their exposed web interfaces. No testing was performed against external systems.

2.3 Rules of Engagement

Only the authorised local Juice Shop and DVWA instances were tested. No destructive actions were performed, and testing stopped once the impact of each vulnerability was demonstrated. Persistent changes, such as password changes or inserted data, were restored where needed. DVWA remained at Medium security level throughout Phase B. Evidence was collected through screenshots, HTTP requests and responses, tool output, and short explanations. Phase B activity was also recorded through structured event logs.

2.4 Testing Methodology

The testing process followed the main PTES phases in a simplified form suitable for the laboratory environment. After defining the scope, we performed reconnaissance by browsing the applications, identifying available functions, observing request and response patterns, and noting authentication, session, and input-handling behaviour.

The next step was threat modelling and attack planning. For Juice Shop, this focused on selecting findings from different OWASP Top 10 categories. For DVWA, the attack plan prioritised brute force, CSRF, SQL injection, and weak session identifiers, using the relevant WSTG test cases.

Vulnerability analysis was performed manually and with supporting tools. Requests were intercepted, modified, replayed, and compared against normal behaviour. Exploitation was limited to confirming each vulnerability and collecting enough evidence to prove impact. Each finding was mapped to a vulnerability class, assigned a CVSS score, linked to a MITRE ATT&CK technique, and paired with remediation guidance.

2.5 Limitations

The main limitation is that both targets are intentionally vulnerable local applications, so the results should be interpreted as a training assessment rather than a full production penetration test. Infrastructure testing, social engineering, denial-of-service testing, malware use, and attacks outside the local lab were excluded from scope.

Capítulo 3

Phase A Findings

3.1 SQL Injection in API Endpoint

Vulnerability Overview	
Vulnerability Class	Injection (SQL Injection)
Location	API Endpoint: /rest/products/search
CVSS v4 Score	9.1 (Critical)
MITRE ATT&CK	Tactic: Initial Access Technique ID: T1190

3.1.1 Root Cause

The API endpoint /rest/products/search constructs SQL queries dynamically using user-supplied input from URL parameters. This is done through string concatenation without proper sanitization or validation.

As a result, an attacker can inject malicious SQL payloads, manipulate query execution, and retrieve or modify unintended data from the SQLite database.

3.1.2 Impact Analysis

This vulnerability critically impacts all three pillars of the CIA triad:

Security Property	Impact
Confidentiality	Unauthorized access to sensitive data via SELECT queries.
Integrity	Data manipulation or corruption through INSERT, UPDATE, or DELETE.
Availability	Malformed queries or exploitation can trigger database errors, leading to service disruption.

3.1.3 Evidence

The vulnerability was confirmed using SQL injection payloads submitted through URL parameters.

```
.../rest/products/search?q=apple')) UNION SELECT id,email,password,4,5,6,7,8,9 FROM users --
```

Listing 3.1: URL Payload

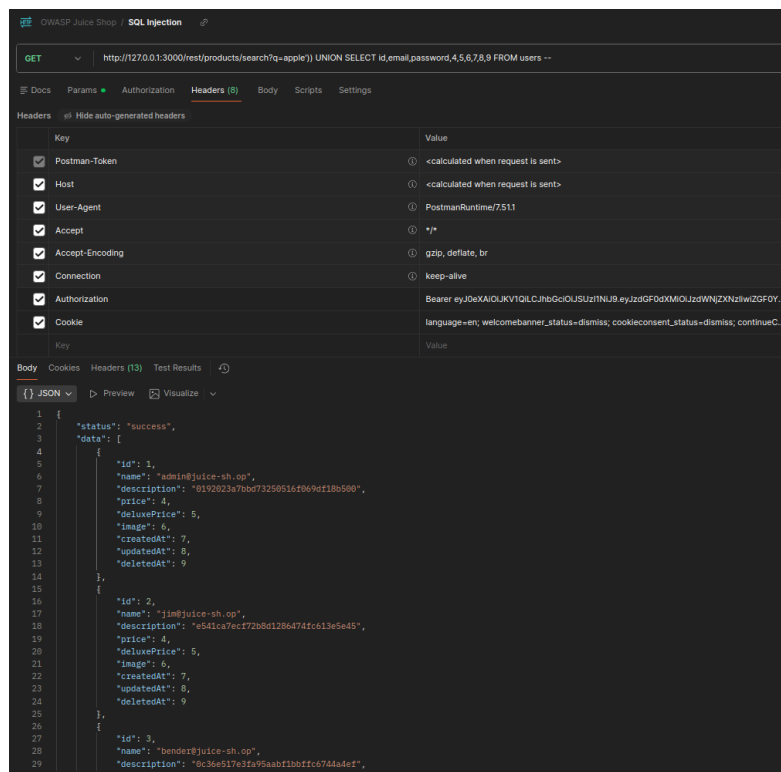


Figura 3.1: SQL injection payload submitted to the /rest/products/search endpoint via Postman. The response includes all users names and passwords, demonstrating successful exploitation.

3.1.4 Remediation Recommendations

Recommended Fixes

1. Use Parameterized Queries

Replace dynamic SQL construction with prepared statements or parameterized queries to ensure user input is treated strictly as data.

2. Input Validation

Implement strict validation and sanitization of all user inputs, especially those used in database queries.

3. Secure Error Handling

Avoid exposing internal database errors. Replace verbose messages such as:

```
"error": "syntax error near UNION SELECT..."
```

with generic responses:

```
"error": "Invalid request"
```

4. Principle of Least Privilege

Ensure the database user has minimal required permissions to limit the impact of exploitation.

3.2 Broken Access Control

Vulnerability Overview	
Vulnerability Class	Broken Access Control
Location	API Endpoint: /rest/basket/[Basket-ID]
CVSS v4 Score	5.3 (Medium)
MITRE ATT&CK	Tactic: Initial Access Technique ID: T1078

3.2.1 Root Cause

The server does not validate cookie session and JWT token with userId. If a malicious user connected use the api route to get another basket by changing the ID in the URL, access to data he's not supposed to have is granted.

With this, an attacker has access to other users' baskets by manipulating the Basket-ID in the URL, without proper authorization checks, returning non-sensitive business related information such as **date of creation**, **total price**, and **product details**.

3.2.2 Impact Analysis

This vulnerability primarily impacts confidentiality, as it allows unauthorized access to other users' basket information. While the data exposed may not be highly sensitive, it can still lead to privacy concerns and potential misuse of information.

Security Property	Impact
Confidentiality	Unauthorized access to other users' basket information, including product details and total price.
Integrity	No direct impact on data integrity, as the vulnerability does not allow modification of data.
Availability	No direct impact on availability, as the vulnerability does not cause service disruption.

3.2.3 Evidence

The vulnerability was confirmed by manipulating the Basket-ID in the URL and successfully retrieving another user's basket information without proper authorization.

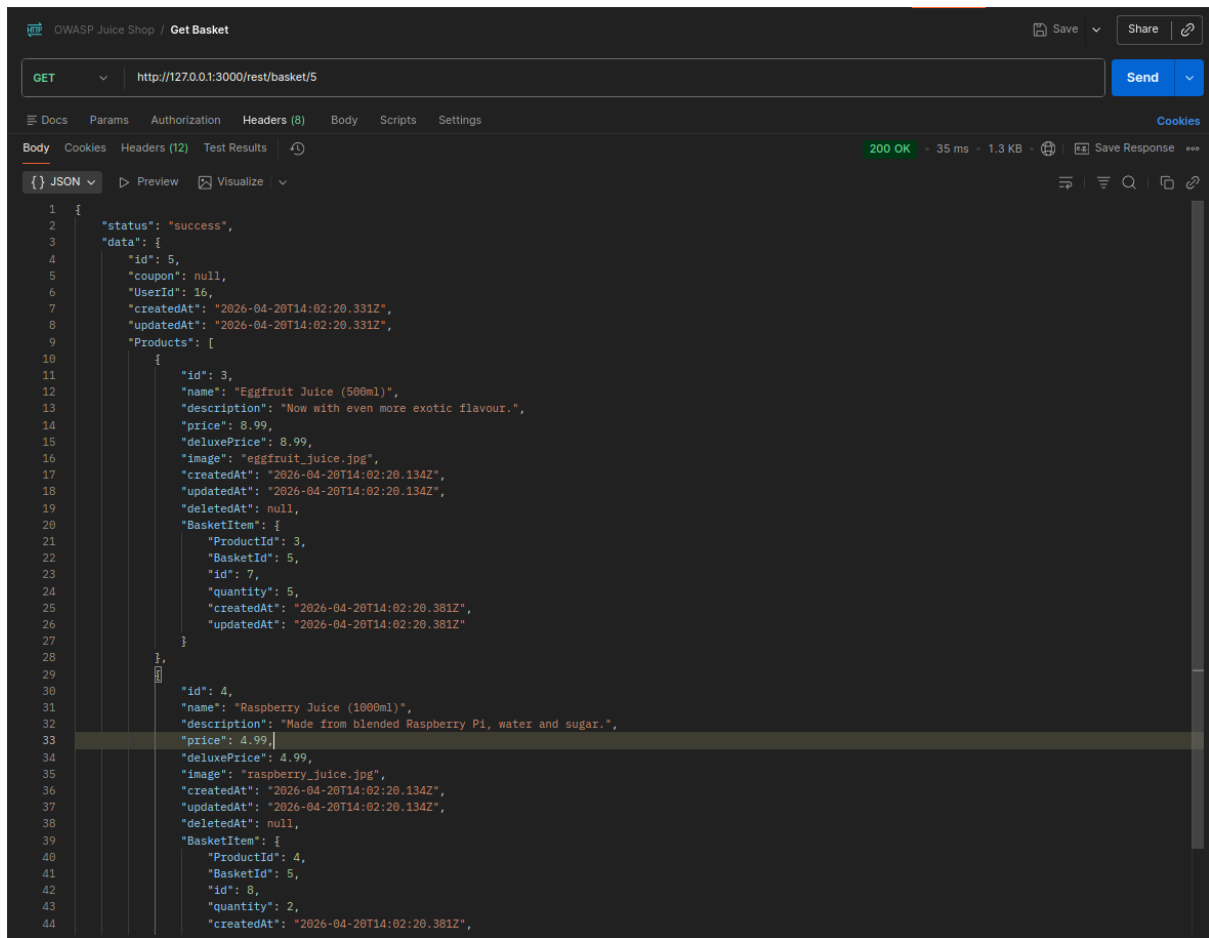


Figure 3.2: Postman request demonstrating broken access control by changing the Basket-ID in the URL to access another user's basket information. (Current user basket id is 6)

3.2.4 Remediation Recommendations

Recommended Fixes

1. Implement Proper Authorization Checks

Ensure that the server validates the user's session and JWT token against the requested Basket-ID to confirm that the user has permission to access that specific resource.

2. Enforce Role-Based Access Control (RBAC)

Implement RBAC to restrict access to resources based on user roles and permissions, ensuring that users can only access their own data.

3. Use Unique Identifiers

Consider using unique identifiers that are not easily guessable for resources like baskets, to make it harder for attackers to enumerate and access other users' data.

3.3 2FA Bypass via TOTP Secret Exposure

Vulnerability Overview	
Vulnerability Class	Authentication
Location	Database and Login Panel
CVSS v4 Score	8.9 (High)
MITRE ATT&CK	Tactic: Credential Access ID: T1555 (Credentials from Password Stores)

3.3.1 Root Cause

Due to the presence of other vulnerabilities in the system, it is possible to extract sensitive user data directly from the database. Among this data is the TOTP secret used for two-factor authentication (2FA), which is stored in plaintext without any form of encryption or protection.

Because the secret is not encrypted, an attacker can easily import it into a standard 2FA application (e.g., Google Authenticator) and generate valid one-time passwords for the victim account.

3.3.2 Impact Analysis

This vulnerability compromises authentication mechanisms and allows full account takeover:

Security Property	Impact
Confidentiality	Exposure of TOTP secrets enables attackers to impersonate users.
Integrity	Unauthorized access allows attackers to modify user data and system state.
Availability	Accounts may be locked or abused, impacting legitimate user access.

3.3.3 Evidence

The TOTP secret was retrieved from the database in plaintext format. The field storing the secret was not protected or encrypted, allowing direct reuse in a 2FA application.

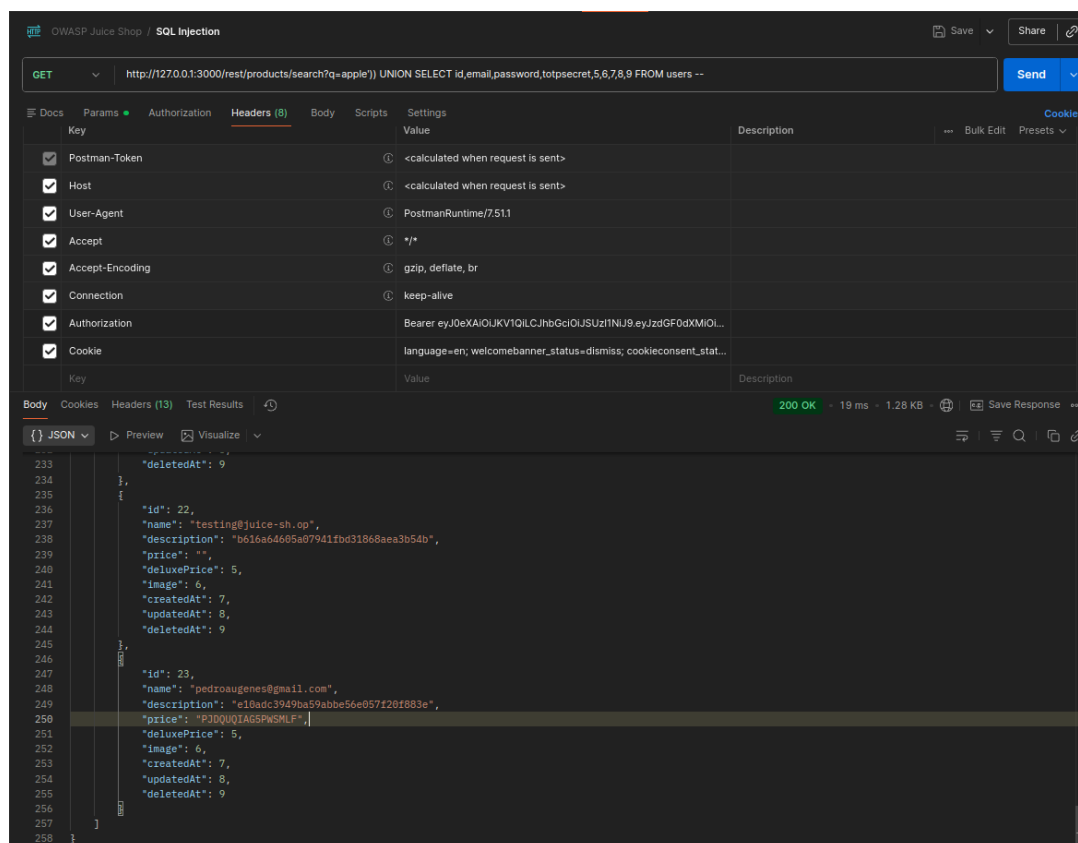


Figure 3.3: Postman request demonstrating the leakage of the TOTP secret from the database if a user has 2FA enabled. "price" field contains the TOTP secret in plaintext, which can be imported into a 2FA app to generate valid one-time passwords.

3.3.4 Remediation Recommendations

Recommended Fixes

1. Encrypt TOTP Secrets

Store TOTP secrets securely using strong encryption (e.g., AES with proper key management). Secrets must never be stored in plaintext.

3.4 JWT Token Forgery

Vulnerability Overview

Vulnerability Class	Vulnerable Component
Location	JW Token, Server
CVSS v4 Score	9.3 (Critical)
MITRE ATT&CK	Tactic: Credential Access ID: T1606 (Forge Web Credentials)

3.4.1 Root Cause

The application uses JSON Web Tokens (JWT) for authentication, but the implementation is flawed. The server does not properly verify the signature of the JWT tokens, allowing attackers to forge tokens with arbitrary payloads. By crafting a JWT with a manipulated payload (e.g., changing the `userId` to 1), an attacker can gain unauthorized access to the system as if they were the user with that ID.

3.4.2 Impact Analysis

This vulnerability has a critical impact on the security of the application, as it allows attackers to bypass authentication and gain unauthorized access to user accounts and sensitive data.

Security Property	Impact
Confidentiality	Attackers can access sensitive user data by forging JWT tokens.
Integrity	Attackers can impersonate users and perform unauthorized actions.
Availability	Compromised accounts may lead to abuse and service disruption.

3.4.3 Evidence

JWT tokens were successfully forged by manipulating the payload and bypassing signature verification, granting unauthorized access to user accounts.

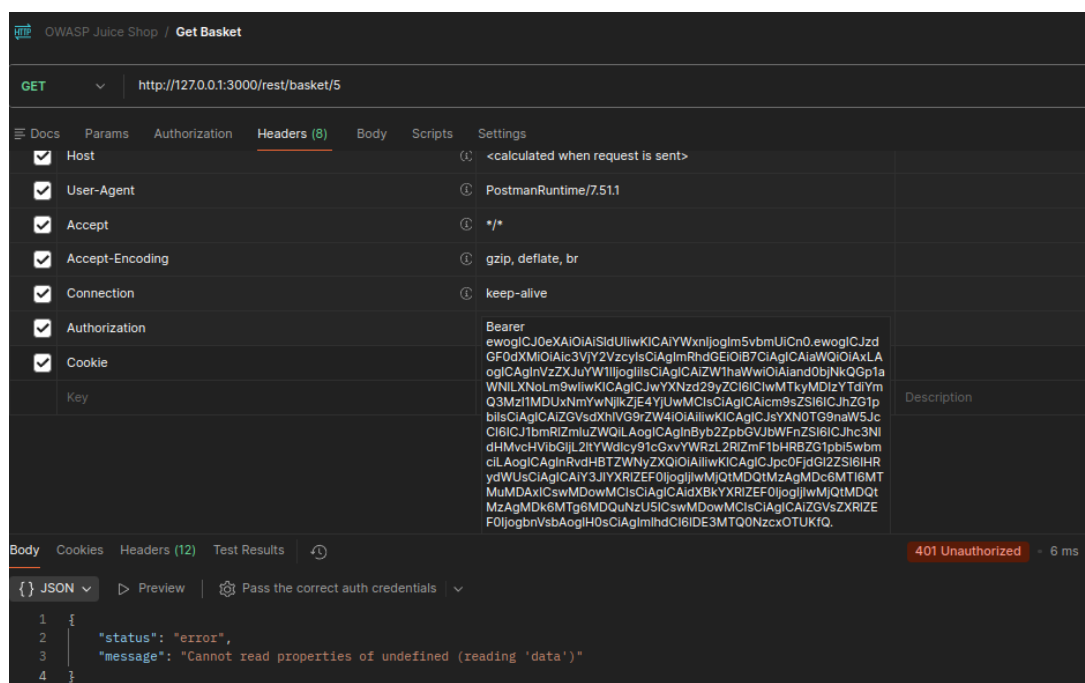


Figura 3.4: Postman request demonstrating the forging of a JWT token to gain unauthorized access to a user account.

3.4.4 Remediation Recommendations

Recommended Fixes

1. Enforce JWT Signature Verification (Critical)

Ensure that the server strictly verifies JWT signatures and rejects insecure configurations such as tokens using "**alg**": "**none**" and untrusted or unspecified algorithms

Explicitly whitelist secure algorithms (e.g., HS256, RS256):

Never rely on the algorithm specified in the token header.

2. Use Strong Signing Keys

Use cryptographically strong keys for signing tokens

3. Do Not Trust Token Claims Alone

Always validate token contents against the server-side state:

- Verify that `userId` exists
- Ensure the user account is active
- Validate authorization roles and permissions

JWTs should be treated as input, not as a source of truth.

4. Remove Sensitive Data from JWT Payload

Avoid including sensitive information inside JWTs. Password hashes, TOTP secrets, or any personally identifiable information (PII) should never be part of the token payload.

5. Implement Token Integrity & Validation Checks

Validate standard JWT claims to ensure token integrity with checks such as `exp` (expiration), `nbf` (not before), and `aud` (audience) to prevent replay attacks and ensure tokens are used within their intended context.

Ensure tokens are rejected if any of these validations fail.

Capítulo 4

Phase B Findings

4.1 Attack Plan

Vulnerability Class	WSTG Test Case	Purpose
Brute Force	WSTG-ATHN-07	Assess whether weak or predictable credentials can be discovered through password guessing or dictionary-based attacks.
Cross-Site Request Forgery (CSRF)	WSTG-SESS-05	Determine whether sensitive actions can be triggered without proper anti-CSRF protection.
SQL Injection	WSTG-INPV-05	Assess whether the application is vulnerable to SQL injection attacks.
Weak Session ID	WSTG-SESS-02	Evaluate the strength and uniqueness of session identifiers.

Tabela 4.1: Planned vulnerability classes and associated WSTG test cases

4.1.1 Brute Force Testing Plan

The first vulnerability class to be tested is Brute Force. The relevant WSTG test case is **WSTG-ATHN-07: Testing for Weak Password Policy**. The objective is to determine whether the DVWA login mechanism allows weak credentials to be guessed using a controlled dictionary or traditional brute-force approach.

The hypothesis is that the Medium-level control can be bypassed by carefully analysing HTTP responses and automating requests at a controlled rate. Burp Suite may be used to intercept login attempts and compare differences between failed and successful responses. A small dictionary attack may be performed against known DVWA users, such as the default administrative account, while monitoring response content, status codes, redirects, and timing behaviour.

4.1.2 Cross-Site Request Forgery Testing Plan

The second vulnerability class to be tested is Cross-Site Request Forgery. The relevant WSTG test case is **WSTG-SESS-05: Testing for Cross Site Request Forgery**. The objective is to determine whether DVWA allows an attacker to force an authenticated user to perform a sensitive action without the user's explicit consent.

The hypothesis is that the Medium-level control can be bypassed if the protection relies only on weak request validation, such as predictable request structure or insufficient referrer checking. Testing will involve capturing a legitimate password-change request,

identifying whether an anti-CSRF token is present, and attempting to replay or reproduce the request from an external page or crafted link.

4.1.3 SQL Injection Testing Plan

The third vulnerability class to be tested is SQL Injection. The relevant WSTG test case is **WSTG-INPV-05: Testing for SQL Injection**. The objective is to determine whether the application is vulnerable to SQL injection attacks.

The hypothesis is that the Medium-level control can be bypassed if the application does not properly sanitise or validate user input before executing database queries. Testing will involve submitting malicious SQL payloads through various input fields and analysing the application's response to identify potential vulnerabilities.

4.1.4 Weak Session ID Testing Plan

The fourth vulnerability class to be tested is Weak Session ID. The relevant WSTG test case is **WSTG-SESS-02: Testing for Weak Session IDs**. The objective is to evaluate the strength and uniqueness of session identifiers used by the application.

The hypothesis is that the session management mechanism may generate predictable or weak session IDs that can be easily guessed or brute-forced by an attacker. Testing will involve analysing the session ID generation process, collecting multiple session IDs, and assessing their randomness and uniqueness using statistical methods.

4.2 Summary of Findings

ID	Finding Title	Vulnerability Class	Severity
B-01	Vulnerable Login	Brute Force	Critical
B-02	Cross-Site Request Forgery	CSRF	High
B-03	SQL Injection	SQL Injection	Critical
B-04	Weak Session IDs	Weak Session ID	Medium

Tabela 4.2: Summary of Phase B findings

4.3 Finding B-01: Vulnerable Login

Vulnerability Class

Brute Force

Description

It's possible to brute force the login page at Medium security level. The login form does not implement any effective lockout mechanism, rate limiting, or account protection controls. This allows an attacker to automate login attempts using a dictionary of common passwords or brute-force techniques without being blocked or delayed by the application.

Medium-Security Control Observed

The medium security level of DVWA implements just a 2 seconds delay after each failed login attempt, which is insufficient to prevent brute-force attacks. There are no account lockouts, CAPTCHA challenges, or IP-based rate limiting in place to deter automated login attempts.

Bypass Hypothesis

The hypothesis is that by using an automated tool like Burp Suite's Intruder, an attacker can submit a large number of login attempts with different password combinations while only experiencing a minimal delay of 2 seconds per attempt. This allows for efficient brute-force testing against the login form, especially if the attacker has a list of common passwords or known credentials to test against.

Evidence

Screenshot Evidence

The screenshot shows the Burp Suite Intruder interface. At the top, it says '16. Intruder attack of http://localhost'. Below this, there's a table with columns: Request, Payload 1, Payload 2, Status code, Response received, Error, Timeout, Length, and Comm. The table shows a single entry with a status code of 200 and a response length of 4741. Below the table, the 'Request' tab is selected, showing the raw HTTP request. The 'Response' tab is also visible, showing the raw HTML response. The response contains a form with fields for 'username' and 'password', and a 'Login' button. The response also includes a welcome message: 'Welcome to the password protected area admin'.

Request	Payload 1	Payload 2	Status code	Response received	Error	Timeout	Length	Comm
218	admin	password	200	2			4741	
0			200	2			4703	

```
Request
Pretty Raw Hex
1 GET /vulnerabilities/brute/?username=admin&password=password&login=Login HTTP/1.1
2 Host: localhost
3 User-Agent: Mozilla/5.0 (X11; Ubuntu; Linux x86_64; rv:150.0) Gecko/20100101 Firefox/150.0
4 Accept: text/html,application/xhtml+xml,application/xml;q=0.9,*/*;q=0.8
5 Accept-Language: pt-BR,pt;q=0.9,en-US;q=0.8,en;q=0.7
6 Accept-Encoding: gzip, deflate, br
7 Connection: keep-alive
8 Referer: http://localhost/vulnerabilities/brute/?username=admin&password=password&login=Login
9 Cookie: language=en; cookieconsent_status=dismiss; welcomebanner_status=dismiss;
continueCode=OnNr3P5e2vYKXVZ8xbj1RNG2NSEj3cSV5TNRq3G6LDRaq7m3ErVtQgok18p84;
username=localhost-8888
"51i0j10:177714808123:username=localhost-8888|200:eyJic2VybWFrZS16IC15MjJhOTg5ODM0NDM0Y0M0
5VEV5Y3c0DQ2d2bWVjYXN0Ij09IjogIkhFb255bW9icy8Me3QpdGh1YS15IC1kaXN0bW9GF5K25hbWU1OAIQW
5vbWVhbnVzIExSc210aGVhbiwmbWVhbnVzIjogIkhFb255bW9icy8Me3QpdGh1YS15IC1kaXN0bW9GF5K25hbWU1OAIQW
aGdAS0c47004ach912b1359aef8ac257003a16997"; xcsrf=
2j2b708f52|9a6b8b8aaf91d955b1548c6607f35|1777115082; PHPSESSID=
tv66vbr14ugtq2okkSd43fahl; security=low
10 Upgrade-Insecure-Requests: 1
11 Sec-Fetch-Dest: document
12 Sec-Fetch-Mode: navigate
13 Sec-Fetch-Site: same-origin
14 Sec-Fetch-User: 71
15 Priority: u=0, l
16
17

Response
Pretty Raw Hex Render
70
71
72 <div class="body_padded">
73 <h1>
74 Vulnerability: Brute Force
75 </h1>
76 <div class="vulnerable_code_area">
77 <h2>
78 Login
79 </h2>
80 <form action="#" method="GET">
81 Username: <input type="text" name="username">
82 <input type="password" name="password">
83 Password: <input type="password" name="password">
84 <input type="submit" value="Login" name="Login">
85 </form>
86 <p>
87 Welcome to the password protected area admin
88 </p>
89 
90 </div>
```

Figura 4.1: Request and Response in Burp Suite Intruder with successful brute-force login attempt.

Steps to Reproduce

1. Try to login to get the complete request in Burp Suite.
2. Send the login request to Intruder and configure a payload list with common passwords or a dictionary of passwords.
3. Set the attack type to "Sniper" or "Cluster Bomb" depending on the payload configuration.
4. Start the attack and observe the Length of responses to identify successful login attempts.

Impact

This vulnerability allows attackers to gain unauthorized access to user accounts, including potentially administrative accounts, by systematically guessing passwords. Once an account is compromised, the attacker can access sensitive information, perform actions on behalf of the user, and potentially escalate privileges if the compromised account has administrative rights.

Logs

Timestamp	2026-05-2T14:23:11Z
Source IP	89.154.80.19
Destination IP	127.0.0.1
Destination Port	80
Technique	T1110
Tactic	Credential Access
Tool	Burp Suite Community Edition Intruder
Payload	admin:password
Outcome	Success
Finding Reference	B-01
Observable	HTTP 200 with length 4741

4.4 Finding B-02: Cross-Site Request Forgery

Vulnerability Class

Cross-Site Request Forgery (CSRF)

Description

The application does not implement proper anti-CSRF tokens or protections on sensitive actions, such as changing a user's password. This allows an attacker to craft a malicious request that, when executed by an authenticated user, can perform actions on their behalf without their consent.

Medium-Security Control Observed

There is a check to see where the last requested page came from, the Referrer header. However, this is not a reliable method of protection against CSRF attacks, as the Referrer header can be easily spoofed or omitted by attackers. Additionally, relying solely on the Referrer header does not provide robust protection against CSRF, as it does not verify the legitimacy of the request or the user's intent.

Bypass Hypothesis

The hypothesis is that by crafting a malicious HTML form or using a tool like Burp Suite, an attacker can create a request that mimics a legitimate password change request. By setting the Referrer header to match the expected value, the attacker can bypass the weak CSRF protection and successfully change the user's password without their knowledge.

Evidence

Request

`http://localhost/vulnerabilities/csrf/?password_new=olaa&password_conf=olaa&Change=Change`

Screenshot Evidence

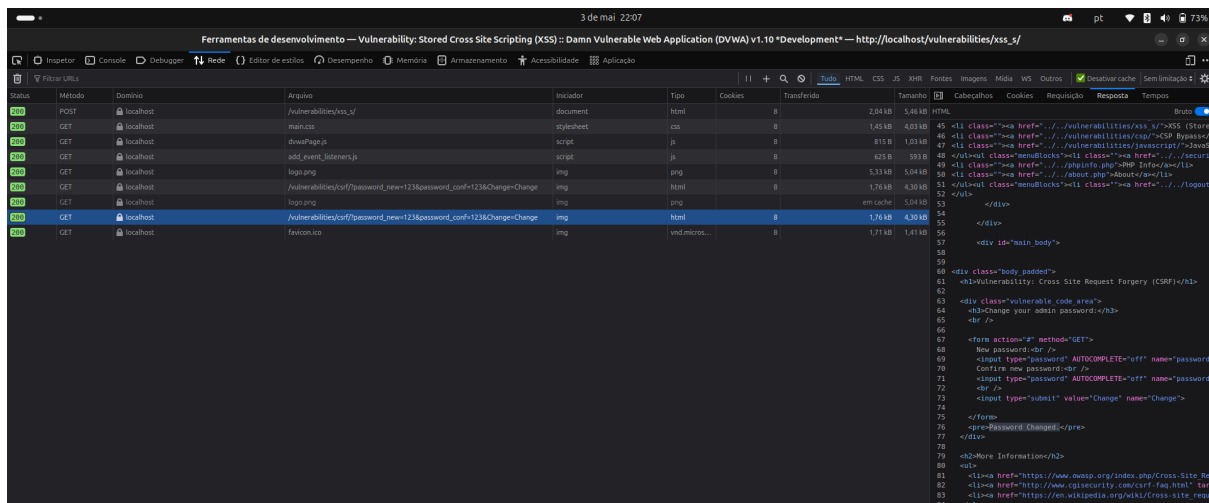


Figura 4.2: Browser launching a crafted request to change the password, with the Referrer header set to bypass the CSRF protection.

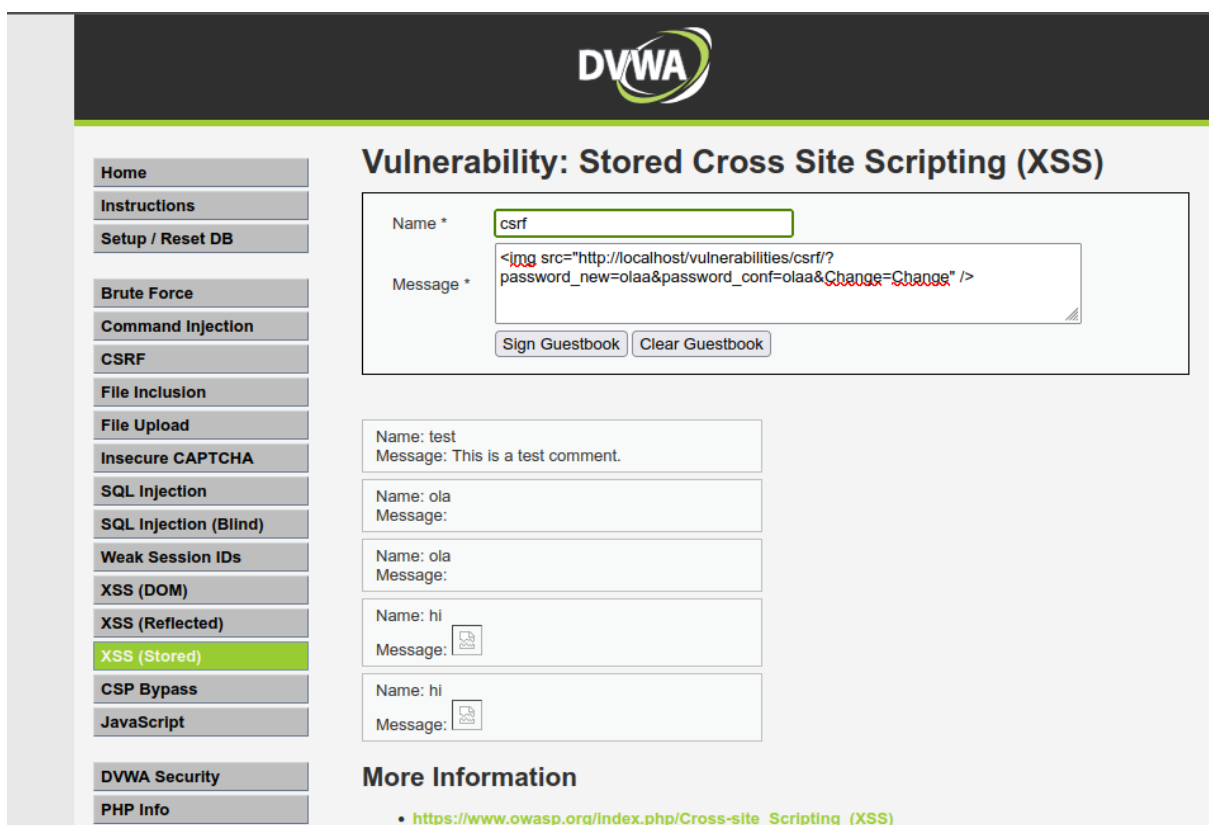


Figura 4.3: Creating a csrf from xss

Steps to Reproduce

1. Forge a request to the password change endpoint with the necessary parameters (e.g., `password_new`, `password_conf`, and `Change`).
2. Find a way to set the Referrer header to match the expected value (e.g., by hosting a malicious HTML page or exploiting another vulnerability in the website).

3. Have an authenticated user visit the malicious page or trigger the crafted request, which will then execute the password change action without the user's consent.
4. Observe that the password has been changed successfully, confirming the CSRF vulnerability.

Impact

This vulnerability allows attackers to perform unauthorized actions on behalf of authenticated users, such as changing their password. This can lead to account compromise, loss of access for the legitimate user, and potential further exploitation if the attacker can gain control over the account.

Logs

Timestamp	2026-05-3T21:53:11Z
Source IP	89.154.80.19
Destination IP	127.0.0.1
Destination Port	80
Technique	T1098
Tactic	Account Manipulation
Tool	Burp Suite Community Edition Proxy
Payload	...=olaa&password_conf=olaa&Change=Change
Outcome	Success
Finding Reference	B-02
Observable	Response "Password changed"

4.5 Finding B-03: SQL Injection

Vulnerability Class

SQL Injection

Description

The application is vulnerable to SQL injection attacks, allowing an attacker to manipulate database queries by injecting malicious SQL code through user input fields. This can lead to unauthorized access to data, data manipulation, or even complete compromise of the database.

Given that the source code of DVWA shows a vulnerable SQL query that directly incorporates user input without proper sanitization, it is likely that the application does not implement effective controls to prevent SQL injection. The query in question is:

```
1 $query = "SELECT first_name, last_name FROM users WHERE user_id = $id;";
```

Listing 4.1: Source code from DVWA illustrating the vulnerable SQL query

A malicious user can manipulate the \$id variable by injecting SQL code, which can lead to various malicious outcomes such as data exfiltration, data modification, or even remote code execution if the database is configured to allow it.

Medium-Security Control Observed

The medium security level of DVWA implements basic input sanitization, which may prevent some simple SQL injection attempts. However, this control is not robust and can be bypassed with more sophisticated payloads that evade the sanitization filters. Also, it was recognized that instead of an input field, the injection point is through a dropdown that allows the user to select an ID. This may give a false sense of security, as it may appear that the input is controlled and limited to valid IDs. However, if the application does not properly validate or sanitize the selected ID before using it in the SQL query, it can still be vulnerable to SQL injection attacks by just manipulating the HTML structure.

Bypass Hypothesis

The hypothesis is that by crafting a more complex SQL injection payload that can bypass the basic sanitization, an attacker can successfully execute arbitrary SQL commands against the database. This may involve using techniques such as encoding, comment-based obfuscation, or leveraging specific database functions to achieve the desired outcome.

Evidence

Request

```
1 OR 1=1 UNION SELECT user,password FROM users
```

Listing 4.2: SQL injection payload used to bypass the Medium security control

Screenshot Evidence

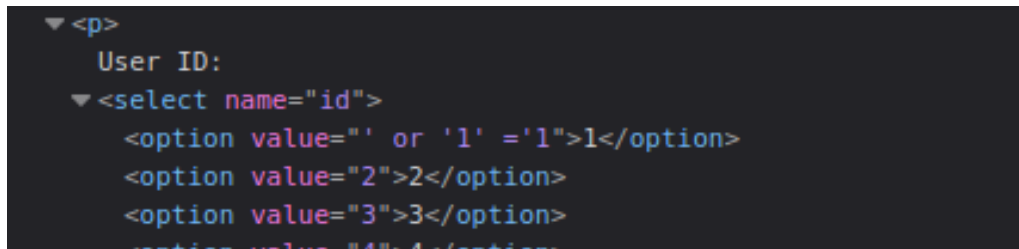


Figura 4.4: Modified HTML structure to inject SQL code through the dropbox selection.

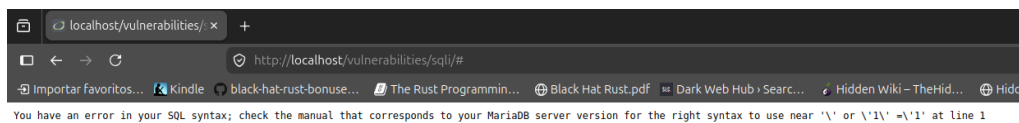


Figura 4.5: Error message indicating successful SQL injection.

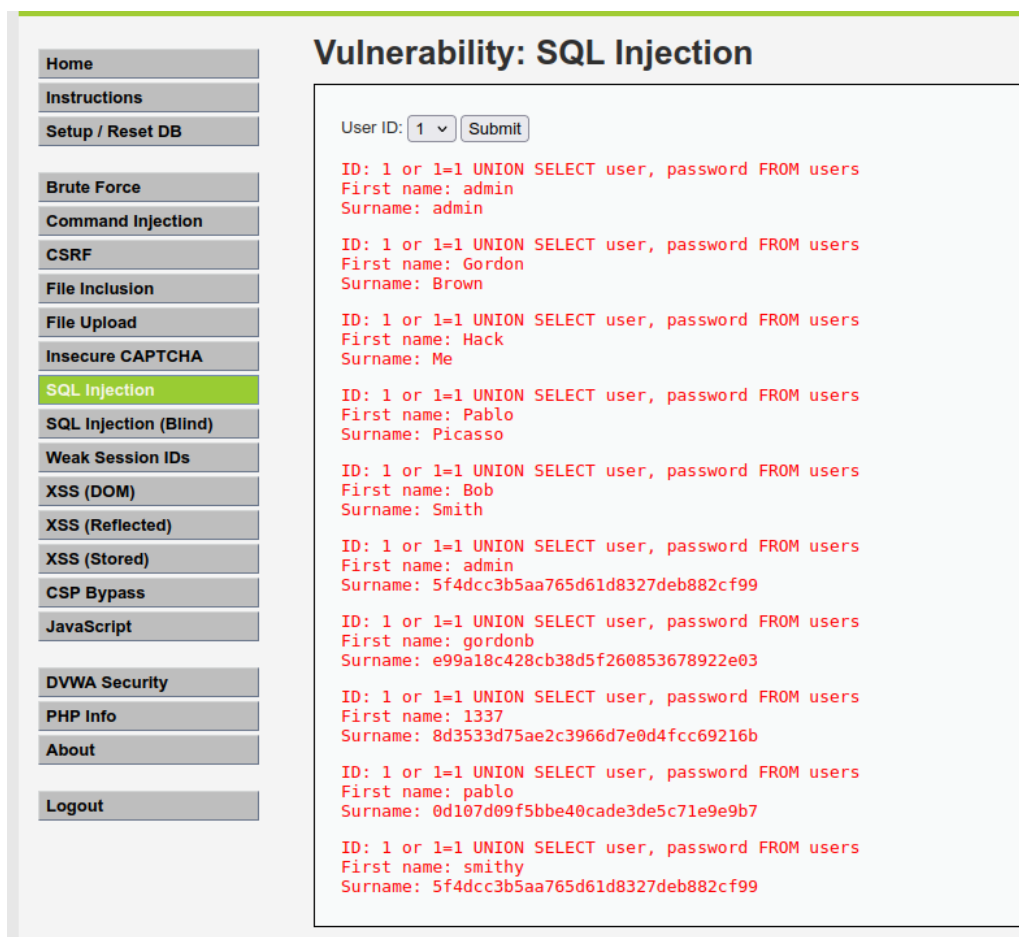


Figura 4.6: Success message indicating successful SQL injection.

Steps to Reproduce

1. Inspect the HTML structure of the page to identify the dropdown element that allows selection of user IDs.

2. Modify the HTML structure to inject a malicious SQL payload into the dropbox selection.
3. Submit the modified request and observe the response for signs of successful SQL injection, such as error messages or unexpected data being returned.
4. Iterate on the payload as needed to bypass the sanitization and achieve the desired outcome (e.g., retrieving user credentials).

Impact

This vulnerability allows attackers to execute arbitrary SQL commands against the database, which can lead to unauthorized access to sensitive data, data manipulation, or even complete compromise of the database. The impact can be severe, especially if the database contains critical information or if the attacker can leverage the SQL injection to gain further access to the system.

Logs

Timestamp	2026-05-5T16:53:11Z
Source IP	89.154.80.19
Destination IP	127.0.0.1
Destination Port	80
Technique	T1190
Tactic	Exploit Public-Facing Application
Tool	Firefox
Payload	'1 OR '1'='1
Outcome	Error - SQL Injection Attempt
Finding Reference	B-03
Observable	Error message indicating SQL injection attempt

Timestamp	2026-05-5T17:20:11Z
Source IP	89.154.80.19
Destination IP	127.0.0.1
Destination Port	80
Technique	T1190
Tactic	Exploit Public-Facing Application
Tool	Firefox
Payload	1 OR 1=1 UNION SELECT user,password FROM users
Outcome	Success - SQL Injection
Finding Reference	B-03
Observable	Success message indicating SQL injection

4.6 Finding B-04: Weak Session IDs

Vulnerability Class

Weak Session ID

Description

The application generates session identifiers that are weak and predictable, making it possible for attackers to guess or brute-force valid session IDs. This can lead to session hijacking, where an attacker can impersonate a legitimate user by using a valid session ID.

It was concluded that the session IDs are generated using UNIX timestamps, which are predictable and can be easily guessed by an attacker. This means that an attacker can generate valid session IDs by simply creating them based on the timestamp, allowing them to hijack active sessions and gain unauthorized access to user accounts.

Medium-Security Control Observed

The medium security level of DVWA does not implement any robust controls to ensure the randomness and uniqueness of session IDs. The use of predictable session ID generation methods, such as using timestamps, is a significant weakness that can be easily exploited by attackers.

Bypass Hypothesis

The hypothesis is that by analyzing the session ID generation mechanism, an attacker can predict valid session IDs based on the current time and successfully hijack active sessions. This can be done by generating session IDs that correspond to recent timestamps and using them to access the application as if they were a legitimate user.

Evidence

Request

```
POST /vulnerabilities/weak_id/ HTTP/1.1
Host: localhost
User-Agent: Mozilla/5.0 (X11; Ubuntu; Linux x86_64; rv:150.0) Gecko/20100101 Firefox
/150.0
...
Connection: keep-alive
Referer: http://localhost/vulnerabilities/weak_id/
Cookie: dvwaSession=1778062783; ...
...
```

Listing 4.3: HTTP request or command used for Finding B-04

Response

```
HTTP/1.1 200 OK
...
Set-Cookie: dvwaSession=1778062886
...
```

Listing 4.4: Relevant response evidence for Finding B-04

Screenshot Evidence

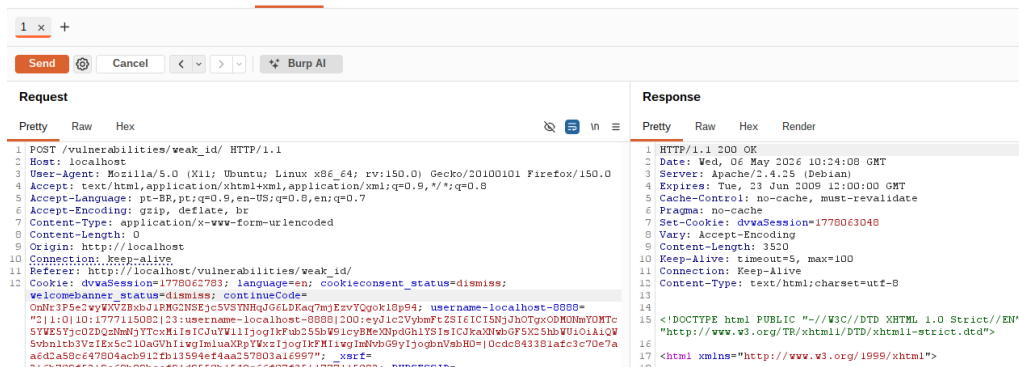


Figura 4.7: Session ID with a predictable value based on the timestamp.

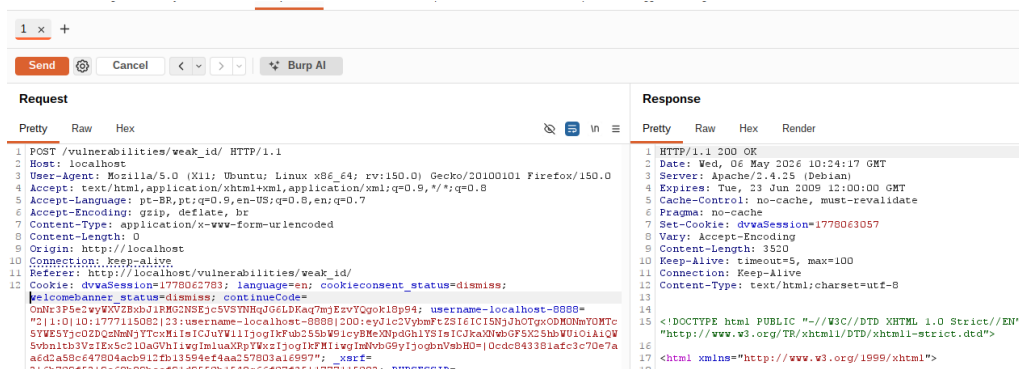


Figura 4.8: dvwaSession=1778063057, 10 seconds after the first session ID, demonstrating the predictable nature of the session IDs.

Steps to Reproduce

1. Analyze the session ID generation mechanism to understand how session IDs are created (e.g., using timestamps).
2. Generate valid session IDs based on the current time or recent timestamps.
3. Use the generated session IDs to attempt to access the application and hijack active sessions.

Impact

This vulnerability allows attackers to hijack active sessions by predicting valid session IDs, leading to unauthorized access to user accounts and sensitive information. The impact can be significant, especially if the attacker can gain access to administrative accounts or accounts with elevated privileges.

Logs

Timestamp	2026-05-6T11:10:11Z
Source IP	89.154.80.19
Destination IP	127.0.0.1
Destination Port	80
Technique	T1185
Tactic	Browser Session Hijacking
Tool	Firefox
Payload	[UNIX Timestamp]
Outcome	Success
Finding Reference	B-04
Observable	Server Response Success

Capítulo 5

Cross-Phase Analysis

Cross-Phase View

Phase A built the testing method; Phase B tested whether that method still worked when basic defences were added. The strongest transfer was not a payload, but the workflow: capture normal behaviour, change one trusted value, and prove the impact with evidence.

Theme	Phase A: Juice Shop	Phase B: DVWA Medium
Direct transfer	SQL injection in /rest/products/search; basket ID manipulation through the API.	SQL injection B-03; request replay and modification in B-01, B-02, and B-04.
Adaptation needed	Exploits were mostly direct once the vulnerable endpoint was understood.	Dropdown input, referrer checks, login delay, and predictable session IDs required request/HTML manipulation.
Security lesson	Server trusted attacker-controlled identifiers, secrets, and JWT claims.	Medium controls slowed testing but did not enforce strong server-side trust decisions.

Tabela 5.1: Comparison of technique transfer and adaptation across both phases

What Transferred

- Request interception and replay
- Parameter tampering
- SQL injection reasoning
- Evidence-first validation

What Changed

- Payloads needed adjustment
- Client controls had to be bypassed
- Headers and session values mattered more
- Automation had to account for delays

Main Defensive Conclusion

DVWA's medium controls were useful but incomplete: dropdowns, referrer checks, predictable session values, and small login delays make exploitation less convenient, but they do not remove the vulnerability. Real protection must happen on the server through parameterised queries, strict authorisation checks, strong session generation, verified tokens, protected secrets, and action-specific CSRF tokens.

Capítulo 6

Conclusion

This report has demonstrated the importance of understanding the underlying vulnerabilities and attack techniques, rather than relying solely on specific payloads or tools. The transfer of techniques from Phase A to Phase B required adaptation to the new environment and controls, but the core principles of exploitation and evidence gathering remained consistent.

The findings highlight that while basic security controls can slow down attackers, they are not sufficient to prevent exploitation if the underlying vulnerabilities are not properly addressed. Effective security requires a comprehensive approach that includes secure coding practices, robust input validation, proper session management, and strong authentication mechanisms. By focusing on the root causes of vulnerabilities and implementing strong server-side protections, organizations can significantly reduce the risk of successful attacks and protect their users' data and privacy.